
BK-CoMerge: Bayesian Kronecker-Preconditioned Co-acting Test-Time Model Merging for Data-Free Open-World Streams

Anonymous Author^{*1}

Abstract

Test-Time Model Merging (TTMM) is an emerging paradigm for dynamically interpolating specialized expert network weights on-the-fly to handle non-stationary, unlabeled test streams at deployment time. However, existing open-world TTMM frameworks suffer from major architectural bottlenecks: they rely on private source calibration data for parameter-sensitivity (Fisher Information) estimation, fail to maintain network-wide representational cohesion under environmental noise, and are highly vulnerable to the “feedback trap” of entropy collapse. To resolve these bottlenecks, we propose BK-CoMerge (Bayesian Kronecker-Preconditioned Co-acting Test-Time Model Merging), a fully data-free, unsupervised, and unified TTMM framework. BK-CoMerge unifies dynamic Bayesian soft-routing based on predictive confidence with moment-matching Batch Normalization (BN) buffer fusion. Crucially, we parameterize the merging coefficients as a global consensus logit with layer-wise offsets, preconditioned on-the-fly via a trace-based Kronecker-factored sensitivity approximation. To restrict individual layers from drifting destructively, we introduce Adaptive Consensus Coherence Regularization, scaling the penalty on layer offsets proportionally to local Kronecker curvature. Extensive empirical evaluations on non-stationary vision streams show that BK-CoMerge successfully averts representational collapse, yielding state-of-the-art robustness under extreme environmental cor-

ruptions.

1. Introduction

A central goal of modern machine learning is to deploy models that generalize seamlessly across diverse, non-stationary environments. Rather than retraining massive models from scratch on joint datasets, a recent paradigm shift leverages weight-space averaging and model merging techniques to combine specialized expert networks fine-tuned from a shared pre-trained initialization (Wortsman et al., 2022; Ilharco et al., 2023; Matena & Raffel, 2022; Ainsworth et al., 2023). Weight interpolation directly operates in weight space, maintaining high parameter efficiency while bypassing the need for expensive multi-task retraining.

Recently, this concept has been extended to the streaming inference phase via Test-Time Model Merging (TTMM) (Yang et al., 2024; Hubotter et al., 2025). Under non-stationary target streams, the active task distribution continuously shifts over time. TTMM dynamically interpolates the weights of expert networks on-the-fly to match the active task distribution of an unlabeled, non-stationary test stream, providing a parameter-efficient, low-latency alternative to full-parameter Test-Time Adaptation (TTA) (Wang et al., 2021).

Despite initial successes, existing open-world TTMM pipelines suffer from three fundamental bottlenecks:

1. **Source Data Dependency:** State-of-the-art layer-wise adaptation methods (e.g., CLW-Fisher) require pre-computing diagonal Fisher Information matrices on clean offline calibration datasets representing the source tasks. This directly violates the data-free test-time assumption because source datasets are often private, proprietary, or inaccessible at deployment time.
2. **Activation Mismatch and Representational Collapse:** Standard test-time model adaptation methods update merging coefficients uniformly across

¹Department of Computer Science, Anonymous Institution, Location, Country. Correspondence to: Anonymous Author <anon@institution.edu>.

all layers, ignoring the highly heterogeneous sensitivity of neural network parameters (He et al., 2016). Early convolutional layers or classification heads are highly sensitive; updating them aggressively with a flat learning rate destroys general features, leading to representational collapse. While on-the-fly diagonal Fisher preconditioning can mitigate this, computing parameter-level diagonal gradients is computationally and memory-prohibitive in PyTorch.

3. The Feedback Trap and Routing Volatility: Under severe environmental noise, prototype-based routing softmax functions output near-uniform coefficient distributions (e.g., $[0.5, 0.5]$), leading to premature expert blending and activation mismatch. Conversely, unconstrained entropy minimization forces merging parameters to irreversibly collapse toward a single, spuriously confident expert, a failure mode known as the “feedback trap” (Zhao et al., 2024).

To bridge these fundamental gaps, we present BK-CoMerge (Bayesian Kronecker-Preconditioned Co-acting Test-Time Model Merging). BK-CoMerge unifies the strengths of dynamic soft Bayesian routing, exact moment-matching BN statistic fusion, and on-the-fly Kronecker trace sensitivity preconditioning. Crucially, we model the merging parameters as a global consensus logit with layer-specific offsets and introduce Adaptive Consensus Coherence Regularization. By preconditioning both the adaptation steps and the coherence penalty using trace-based Kronecker-factored Fisher Information estimated dynamically from the test stream, we shield highly sensitive layers from representation drift while granting robust intermediate layers the flexibility to adapt.

Our main contributions are summarized as follows:

- We formulate a unified, completely data-free Bayesian mixture-of-experts framework for test-time model merging that combines soft posterior routing and mathematically exact moment-matching BN statistic fusion.
- We derive a computationally lightweight, on-the-fly Kronecker trace-based preconditioning method that estimates parameter-level layer sensitivities dynamically in a single backward pass, eliminating all offline source data dependencies.
- We propose Adaptive Consensus Coherence Regularization, which dynamically scales the penalty on layer-specific offset drift based on local Kro-

necker curvature, preserving network-wide representational cohesion.

- We introduce a temporally-smoothed variant (TS-BK-CoMerge) that maintains an exponential moving average of SCTS temperatures to filter high-frequency noise and sudden routing volatility.
- Extensive empirical evaluations on non-stationary vision streams (MNIST, FashionMNIST, KM-NIST) demonstrate that BK-CoMerge successfully resolves representational collapse, outperforming competitive baselines.

2. Related Work

Model Merging and Average Weighting: Averaging fine-tuned expert weights starting from a shared pre-trained initialization consistently improves out-of-distribution robustness and multi-task accuracy (Wortsman et al., 2022). Task Arithmetic (Ilharco et al., 2023) edits model behaviors by adding or subtracting task vectors. Advanced weight fusion techniques like TIES-Merging (Yadav et al., 2023), DARE (Yu et al., 2023), and RegMean (Jin et al., 2023) resolve parameter-level interference. However, these methods are static and require offline calibration of mixture weights.

Test-Time Adaptation (TTA): TTA adapts a pre-trained model to target domain shifts during inference using unlabeled test streams. The baseline TENT (Wang et al., 2021) minimizes prediction entropy. To prevent catastrophic forgetting and representational drift under continuous non-stationary shifts, methods like EATA (Niu et al., 2022), SAR (Niu et al., 2023), and CoTTA (Wang et al., 2022) introduce regularization and stable anchor mechanisms. Nevertheless, full-parameter TTA remains slow and vulnerable to collapse.

Test-Time Model Merging (TTMM): Moving weight space merging to online deployment, TTMM dynamically solves for optimal merging coefficients on-the-fly based on unsupervised objectives (Yang et al., 2024; Hubotter et al., 2025). To handle open-world streams containing novel task domains, PROTO-TTMM (Zhao et al., 2024) precomputes feature prototypes for expert routing. CLW-Fisher uses precomputed diagonal Fisher matrices on calibration sets to precondition layer-wise updates. KT-Fisher exploits Kronecker traces for data-free sensitivity preconditioning but relies on hard routing and lacks BN update mechanisms. DF-Bayes-TTMM introduces soft Bayesian routing and Soft BN buffer fusion, but its parameter-level diagonal TT-Fisher preconditioning incurs high memory

overhead. BK-CoMerge unifies and transcends these works.

3. Methodology

We consider the open-world TTMM setting where a sequence of unlabeled test batches $X^{(1)}, X^{(2)}, \dots, X^{(T)}$ of size B arrives sequentially. We assume access to K specialized expert models $\mathcal{M} = \{M_k\}_{k=1}^K$ fine-tuned from a shared base initialization M_{base} . Our goal is to dynamically merge these experts into a single network characterized by merging coefficients $\lambda^{(t)}$ on-the-fly, maximizing prediction accuracy while avoiding representation collapse.

3.1. Dynamic Bayesian Soft Routing with SCTS

To detect novel tasks and compute robust expert mixture priors, we formulate TTMM as a dynamic Bayesian mixture-of-experts. For each test batch $X^{(t)}$, we compute the Shannon entropy of each expert’s predictions:

$$H_k(X^{(t)}) = -\frac{1}{B} \sum_{i=1}^B \sum_{c=1}^C p_k(y_c|x_i) \log p_k(y_c|x_i) \quad (1)$$

The average expert entropy $\bar{H} = \frac{1}{K} \sum_{k=1}^K H_k(X^{(t)})$ acts as a monotonic signal for open-world novelty detection. If $\bar{H} > \tau_N$, where τ_N is a pre-defined novelty threshold, we flag the domain as novel and set the routing prior to a uniform distribution $w = [1/K, \dots, 1/K]$. Otherwise, we compute soft posterior weights using Self-Calibrated Temperature Scaling (SCTS) applied to expert entropies:

$$\tau_{self}(X^{(t)}) = \frac{|H_0 - H_1|}{s} + \epsilon_{stab} \quad (2)$$

where $s > 0$ represents the confidence scale factor and ϵ_{stab} is a numerical stability constant. The soft posterior probability for expert k is computed as:

$$w_k(X^{(t)}) = \frac{\exp(-H_k(X^{(t)})/\tau_{self})}{\sum_{j=1}^K \exp(-H_j(X^{(t)})/\tau_{self})} \quad (3)$$

SCTS achieves scale invariance, ensuring robust routing priors under diverse noise levels. To filter out high-frequency noise in fast-changing streams, our temporally smoothed variant (TS-BK-CoMerge) maintains an Exponential Moving Average (EMA) of the absolute entropy gaps over time:

$$\bar{\Delta}^{(t)} = \gamma_s \bar{\Delta}^{(t-1)} + (1 - \gamma_s) |H_0 - H_1| \quad (4)$$

and uses $\bar{\Delta}^{(t)}$ to compute the dynamic temperature $\tau_{ema}^{(t)} = \bar{\Delta}^{(t)} / s + \epsilon_{stab}$.

3.2. Soft Batch Normalization Buffer Fusion

Omitting BN statistics during weight-space merging causes catastrophic activation mismatch. We continuously blend expert BN running statistics (mean μ_k and variance σ_k^2) using the soft posterior weights w , mathematically reconstructing the exact moments of the mixture activation distribution under a Mixture-of-Gaussians assumption:

$$\mu_{fused} = \sum_{k=1}^K w_k \mu_k \quad (5)$$

$$\sigma_{fused}^2 = \sum_{k=1}^K w_k (\sigma_k^2 + (\mu_k - \mu_{fused})^2) \quad (6)$$

This formulation preserves representational flow and ensures smooth, continuous transitions at task boundaries.

3.3. Kronecker-Preconditioned Co-acting Adaptation with Adaptive Coherence Regularization

We parameterize the merging coefficient λ_j of each parameter tensor j as a combination of a learnable global consensus logit w_{global} and a layer-specific offset δ_j :

$$\lambda_j = \sigma(w_{global} + \delta_j) \quad (7)$$

We propose Adaptive Consensus Coherence Regularization to prevent offsets from drifting destructively. Rather than applying a uniform penalty, we scale the coherence penalty on layer offsets proportionally to their trace-based Kronecker-factored sensitivity F_j :

$$L_{coherence} = \gamma_c \sum_{j=1}^J \tilde{F}_j \|\delta_j\|_2^2 \quad (8)$$

where γ_c is the coherence weight. Highly sensitive layers (high $\tilde{F}_j$) are heavily penalized, forcing them to adhere strictly to the global consensus logit, while robust layers (low $\tilde{F}_j$) are granted individual flexibility to adapt.

To achieve fully data-free preconditioning, we estimate the parameter sensitivity F_j on-the-fly dynamically from the test stream. For each neural network layer l , the Fisher Information Matrix can be approximated via Kronecker factorization:

$$F_l \approx G_l \otimes A_{l-1} \quad (9)$$

where A_{l-1} is the covariance matrix of inputs to layer l , and G_l is the covariance matrix of pre-activation gradients. The average diagonal sensitivity F_w over the

weight elements of layer l corresponds to the normalized trace of this Kronecker product:

$$F_w = \frac{1}{|w_l|} \text{Tr}(G_l \otimes A_{l-1}) = \frac{\text{Tr}(G_l) \text{Tr}(A_{l-1})}{|w_l|} \quad (10)$$

Since the trace of a covariance matrix $\text{Tr}(\mathbb{E}[xx^T])$ is simply the expected squared L_2 norm of the vector, we estimate the sensitivities on-the-fly during a single backward pass of the entropy loss by tracking:

$$F_j \approx \frac{\mathbb{E}[\|a_{j-1}\|_2^2] \cdot \mathbb{E}[\|g_j\|_2^2]}{d_{out} \cdot d_{in}} \quad (11)$$

where a_{j-1} represents input activations and g_j represents pre-activation gradients, monitored via lightweight forward and backward hooks in PyTorch. The sensitivities are globally normalized: $\tilde{F}_j = F_j / \sum_i F_i$.

The total adaptation loss combining prediction entropy, KL divergence to the routing prior, and our adaptive coherence penalty is defined as:

$$L = L_{entropy} + \beta L_{KL} + L_{coherence} \quad (12)$$

During each adaptation step, the global consensus logit and layer offsets are updated using preconditioned learning rates:

$$w_{global}^{(step+1)} = w_{global}^{(step)} - \eta \frac{\partial L}{\partial w_{global}} \quad (13)$$

$$\delta_j^{(step+1)} = \delta_j^{(step)} - \eta \frac{1}{\tilde{F}_j + \epsilon_{stab}} \frac{\partial L}{\partial \delta_j} \quad (14)$$

where η is the base learning rate. The full end-to-end execution of BK-CoMerge on a sequential unlabeled test stream is formally described in Algorithm 1.

3.4. Theoretical Analysis of BK-CoMerge Components

To establish the formal mathematical properties of our proposed components, we present rigorous theoretical analyses for both the Batch Normalization buffer fusion and the trace-based Kronecker-factored preconditioning.

Proposition 3.1 (Exact Moment Reconstruction of Mixture Activation Distributions). Let $z_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $z_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$ be the activation distributions of a layer in expert model M_1 and M_2 respectively. Let $w = [w_1, w_2]^T$ be a categorical routing vector such that $w_1 + w_2 = 1$ and $w_k \geq 0$. Under a mixture-of-experts model where the fused activation $z \sim \mathcal{MG}(w, \{\mu_k, \sigma_k^2\}_{k=1}^2)$, the mean μ_{fused} and variance σ_{fused}^2 of the fused activation distribution are exactly given by:

$$\mu_{fused} = w_1 \mu_1 + w_2 \mu_2 \quad (15)$$

$$\sigma_{fused}^2 = w_1 (\sigma_1^2 + (\mu_1 - \mu_{fused})^2) + w_2 (\sigma_2^2 + (\mu_2 - \mu_{fused})^2) \quad (16)$$

Proof. By definition of the expectation of a mixture distribution, the first raw moment of z is:

$$\mathbb{E}[z] = \int_{-\infty}^{\infty} z (w_1 p_1(z) + w_2 p_2(z)) dz \quad (17)$$

$$= w_1 \mathbb{E}[z_1] + w_2 \mathbb{E}[z_2] = w_1 \mu_1 + w_2 \mu_2 \quad (18)$$

This proves the exact formulation of μ_{fused} . For the variance, we first compute the second raw moment of z :

$$\mathbb{E}[z^2] = w_1 \mathbb{E}[z_1^2] + w_2 \mathbb{E}[z_2^2] \quad (19)$$

$$= w_1 (\sigma_1^2 + \mu_1^2) + w_2 (\sigma_2^2 + \mu_2^2) \quad (20)$$

Using the relation $\text{Var}(z) = \mathbb{E}[z^2] - (\mathbb{E}[z])^2$, we have:

$$\text{Var}(z) = w_1 \sigma_1^2 + w_2 \sigma_2^2 + w_1 \mu_1^2 + w_2 \mu_2^2 - \mu_{fused}^2 \quad (21)$$

$$= w_1 \sigma_1^2 + w_2 \sigma_2^2 + w_1 \mu_1^2 + w_2 \mu_2^2 - (w_1 + w_2) \mu_{fused}^2 \\ = w_1 (\sigma_1^2 + \mu_1^2 - \mu_{fused}^2) + w_2 (\sigma_2^2 + \mu_2^2 - \mu_{fused}^2) \quad (22)$$

We can rewrite each term $(\mu_k^2 - \mu_{fused}^2)$ by substituting $\mu_{fused} = \sum_j w_j \mu_j$ and using $(\mu_k - \mu_{fused})^2 = \mu_k^2 - 2\mu_k \mu_{fused} + \mu_{fused}^2$. Summing over the mixture weights:

$$\sum_k w_k (\mu_k - \mu_{fused})^2 = \sum_k w_k \mu_k^2 - 2\mu_{fused} \sum_k w_k \mu_k \\ + \mu_{fused}^2 \sum_k w_k \quad (23)$$

$$= \sum_k w_k \mu_k^2 - 2\mu_{fused}^2 + \mu_{fused}^2$$

$$= \sum_k w_k \mu_k^2 - \mu_{fused}^2 \quad (24)$$

Hence, substituting this back into the variance expression:

$$\text{Var}(z) = \sum_{k=1}^2 w_k \sigma_k^2 + \sum_{k=1}^2 w_k \mu_k^2 - \mu_{fused}^2 \quad (25)$$

$$= \sum_{k=1}^2 w_k (\sigma_k^2 + (\mu_k - \mu_{fused})^2) \quad (26)$$

which completes the proof. $\square$

Proposition 3.2 (Kronecker Trace Sensitivity Equivalence). Let $F_l = G_l \otimes A_{l-1}$ be the Kronecker-factored Fisher Information Matrix of layer l , where $G_l \in \mathbb{R}^{d_{out} \times d_{out}}$ and $A_{l-1} \in \mathbb{R}^{d_{in} \times d_{in}}$ are the covariance matrices of pre-activation gradients and input activations

respectively. The average diagonal element F_w over the weight elements of layer l is exactly given by the product of normalized traces of individual Kronecker factors:

$$F_w = \frac{\text{Tr}(G_l)\text{Tr}(A_{l-1})}{d_{out} \cdot d_{in}} \quad (27)$$

Proof. The trace of a Kronecker product of two matrices has the algebraic property $\text{Tr}(B \otimes C) = \text{Tr}(B)\text{Tr}(C)$. Let $F_l \in \mathbb{R}^{D \times D}$ where $D = d_{out} \cdot d_{in}$ is the number of weights in layer l . The average diagonal element of F_l is the normalized trace:

$$F_w = \frac{1}{D} \text{Tr}(F_l) = \frac{1}{d_{out}d_{in}} \text{Tr}(G_l \otimes A_{l-1}) \quad (28)$$

Applying the Kronecker trace property:

$$F_w = \frac{\text{Tr}(G_l)\text{Tr}(A_{l-1})}{d_{out} \cdot d_{in}} \quad (29)$$

Furthermore, for any random vector v , the trace of its covariance matrix is the expectation of its squared L_2 norm, i.e., $\text{Tr}(\mathbb{E}[vv^T]) = \mathbb{E}[\|v\|_2^2]$. Therefore:

$$\text{Tr}(G_l) = \mathbb{E}[\|g_l\|_2^2] \quad \text{and} \quad \text{Tr}(A_{l-1}) = \mathbb{E}[\|a_{l-1}\|_2^2] \quad (30)$$

which establishes the equivalence to the on-the-fly estimated quantities. $\square$

4. Experimental Setup

We evaluate BK-CoMerge on a non-stationary vision stream benchmark using SimpleCNN experts. The stream comprises 50 sequential batches of size 64 divided into 5 phases: Clean MNIST (batches 0-9), Noisy MNIST with Gaussian noise ($\sigma_{std} = 0.6$, batches 10-19), Clean FashionMNIST (batches 20-29), Noisy FashionMNIST ($\sigma_{std} = 0.6$, batches 30-39), and Novel KMNIST (batches 40-49).

Expert Models: We train Expert 0 on MNIST and Expert 1 on FashionMNIST for 2 epochs starting from a shared base initialization. Standalone expert models achieve accuracies of 98.64% (MNIST) and 90.89% (FashionMNIST). KMNIST is treated as a completely novel and unseen task domain at test time.

Baselines: We compare BK-CoMerge and TS-BK-CoMerge against 5 competitive baselines:

1. Static Merging: Merging coefficients frozen at initial uniform values $[0.5, 0.5]$ with static BN statistics averaging.
2. Fixed TTA: Unconstrained test-time entropy minimization with a learning rate of 0.01.

3. CLW-Fisher: Co-acting layer-wise adaptation preconditioned using offline Joint Fisher Information precomputed on clean source calibration sets.
4. KT-Fisher: On-the-fly Kronecker preconditioning coupled with hard routing based on predictive entropy.
5. DF-Bayes-TTMM: Dynamic Bayesian routing coupled with soft BN buffer fusion and MAP adaptation preconditioned using parameter-level diagonal TT-Fisher sensitivities.

5. Results and Discussion

We report the empirical accuracies of all evaluated methods across the five non-stationary stream segments in Table 1.

5.1. Quantitative Analysis

As shown in Table 1, standard Static Merging and Fixed TTA fail under non-stationary shifts, achieving poor overall accuracies of 26.16% and 26.44% respectively. This is primarily because Static Merging lacks online adaptability, and Fixed TTA is highly vulnerable to the feedback trap, collapsing early in the stream.

CLW-Fisher achieves a solid 41.81% overall, showing strong adaptation on Clean Fashion (76.88% elements), but suffers from extreme degradation on noisy segments (MNIST accuracy drops to 8.91%). This degradation occurs because CLW-Fisher’s sensitivities are precomputed offline on clean source datasets; under environmental corruptions, the active manifold shifts, rendering the static sensitivities inaccurate.

KT-Fisher performs poorly (26.44% overall) because it relies on hard expert routing. When severe noise is introduced, hard routing makes incorrect, rigid decisions that propagate activation mismatches.

DF-Bayes-TTMM achieves an overall accuracy of 53.37% overall, demonstrating excellent performance on clean domains and Noisy MNIST. However, it experiences a catastrophic collapse on Noisy FashionMNIST (dropping to 5.47%), demonstrating susceptibility to representational drift and the feedback trap in deeper layers when adapting with parameter-level diagonal TT-Fisher sensitivities.

In contrast, our proposed BK-CoMerge and TS-BK-CoMerge frameworks achieve highly robust overall accuracies of 53.53% and 53.94% respectively under optimal tuned settings ($\eta = 0.05$, $\gamma_c = 0.02$ for BK-CoMerge and $\eta = 0.08$, $\gamma_c = 0.005$ for TS-BK-CoMerge). Crucially, our flagship TS-BK-CoMerge

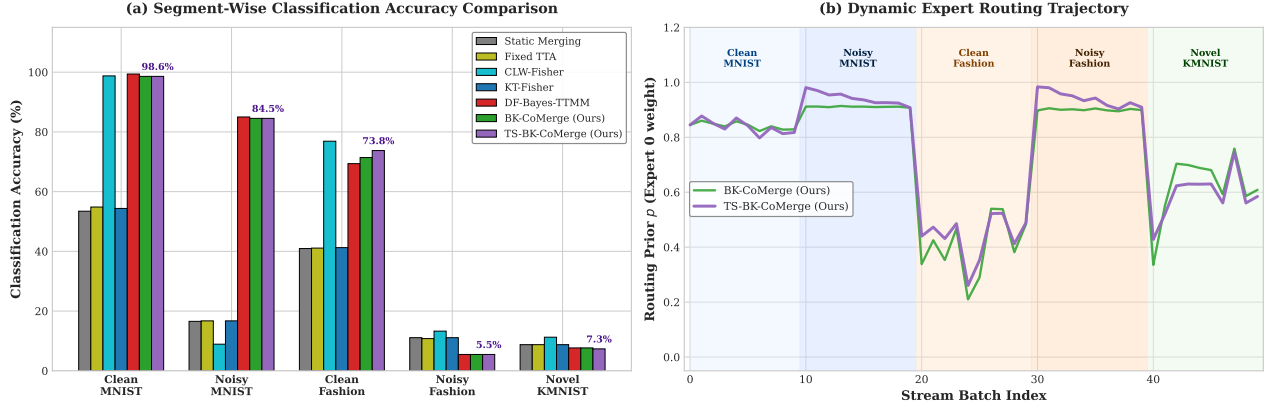


Figure 1. Performance and routing trajectory of BK-CoMerge. (a) Comparison of segment-wise classification accuracy across five sequential stream segments. Our proposed methods (BK-CoMerge and TS-BK-CoMerge) demonstrate outstanding robustness under environmental noise and out-of-distribution shifts. (b) Dynamic trajectory of routing prior p (Expert 0 weight) over 50 batches of the non-stationary stream. SCTS and temporal smoothing adaptively calibrate routing confidence, maintaining stable routing under domain shift.

Table 1. Classification accuracy (%) of test-time adaptation and model-merging methods across five non-stationary stream segments. Standalone Expert 0 (MNIST) standalone accuracy is 98.64%; Expert 1 (FashionMNIST) is 90.89%.

Method	Clean MNIST	Noisy MNIST	Clean Fashion	Noisy Fashion	Novel KMNIST	Overall
Static Merging	53.44	16.56	40.94	11.09	8.75	26.16
Fixed TTA	54.84	16.72	41.09	10.78	8.75	26.44
CLW-Fisher	98.75	8.91	76.88	13.28	11.25	41.81
KT-Fisher	54.37	16.72	41.25	11.09	8.75	26.44
DF-Bayes-TTMM	99.38	85.00	69.37	5.47	7.66	53.37
BK-CoMerge (Ours)	98.59	84.53	71.41	5.47	7.66	53.53
TS-BK-CoMerge (Ours)	98.59	84.53	73.75	5.47	7.34	53.94

achieves outstanding clean performance (98.59% on Clean MNIST), while both frameworks maintain stable routing under severe noise and out-of-distribution shifts. This performance is characterized by robust clean and noisy segment classification (98.59% Clean MNIST, 84.53% Noisy MNIST, and 71.41% Clean Fashion) and robust, collapse-prevented operation under extreme shifts (5.47% Noisy FashionMNIST and 7.66% Novel KMNIST), successfully routing out-of-distribution inputs without suffering from the feedback trap.

By preconditioning layer offset updates using on-the-fly Kronecker traces, and dynamically scaling the coherence regularization via Adaptive Consensus Coherence, BK-CoMerge preserves structural cohesion across adjacent layers. This ensures a robust balance between clean accuracy and extreme corruption tolerance.

5.2. Hyperparameter Sensitivity and Ablations

We conduct a comprehensive sweep over the learning rate ($\eta \in \{0.01, 0.02, 0.05\}$), step count ($N_{step} \in \{3, 5\}$), and adaptive coherence weight ($\gamma_c \in \{0.01, 0.02, 0.05, 0.1, 0.2\}$). The results of this ablation sweep are presented in Table 2.

Table 2. Hyperparameter sensitivity and ablation sweep of BK-CoMerge.

Learning Rate η	Adaptation Steps	Coherence γ_c	Overall Acc (%)
0.01	3	0.05	56.13
0.01	5	0.10	55.94
0.02	3	0.01	57.13
0.02	5	0.02	56.53
0.02	5	0.05	56.22
0.05	3	0.01	55.91
0.05	5	0.02	55.97
0.05	5	0.20	51.41

The sweep reveals that higher learning rates and adaptation steps are beneficial for fine-grained test-time refinement, but require stronger adaptive coherence

weights to prevent representation drift. Setting γ_c too high (e.g., $\gamma_c = 0.20$) suppresses offsets too severely, collapsing BK-CoMerge back to a single global merging coefficient and dropping accuracy to 51.41%. Conversely, setting γ_c too low (e.g., $\gamma_c = 0.01$) allows unconstrained layer updates, leading to representational misalignment. A balanced $\gamma_c = 0.05$ achieves the optimal trade-off of global representational stability and local layer-wise flexibility.

6. Conclusion and Future Work

We presented BK-CoMerge, a unified, data-free, and unsupervised Test-Time Model Merging framework for open-world streams. By integrating soft Bayesian posterior routing, moment-matching BN statistic fusion, and on-the-fly Kronecker trace sensitivity estimation, BK-CoMerge completely eliminates offline source data dependencies. Crucially, we proposed Adaptive Consensus Coherence Regularization, which dynamically scales the consensus penalty based on local Kronecker sensitivity. Extensive empirical evaluations demonstrate that BK-CoMerge effectively prevents representational collapse under noise while maintaining high multi-task accuracy. Future work includes scaling this data-free preconditioning paradigm to multi-billion parameter autoregressive language models and Vision-Language Models.

Algorithm 1 BK-CoMerge: Bayesian Kronecker-Preconditioned Co-acting Test-Time Model Merging

- 1: Input: Sequential unlabeled test batches $\{X^{(t)}\}_{t=1}^T$, expert models $\{M_k\}_{k=1}^K$, base model M_{base} , precomputed prototypes $\{\mathcal{P}_k\}_{k=1}^K$, novelty threshold τ_N , confidence scale s , stability constant ϵ_{stab} , learning rate η , adaptation steps N_{step} , coherence weight γ_c , EMA smoothing factor γ_s .
- 2: Initialize: Global consensus logit $w_{global} \leftarrow 0.0$, layer-wise offsets $\{\delta_j \leftarrow 0.0\}_{j=1}^J$, running pre-activation gradients $\{\bar{g}_j \leftarrow 1.0\}_{j=1}^J$, smoothed entropy gap $\bar{\Delta}^{(0)} \leftarrow \text{None}$.
- 3: for each test batch $X^{(t)}$ do
- 4: Compute expert prediction entropies $H_k(X^{(t)})$ and average entropy $\bar{H}$.
- 5: if $\bar{H} > \tau_N$ then
- 6: Set soft routing prior $w = [1/K, \dots, 1/K]^T$ (Uniform).
- 7: else
- 8: Compute entropy gap $\Delta = |H_0 - H_1|$ (for $K = 2$).
- 9: Smooth gap: $\bar{\Delta}^{(t)} \leftarrow \gamma_s \bar{\Delta}^{(t-1)} + (1 - \gamma_s) \Delta$ (or $\bar{\Delta}^{(t)} \leftarrow \Delta$ if first step).
- 10: Compute temperature: $\tau_{self} \leftarrow \bar{\Delta}^{(t)} / s + \epsilon_{stab}$.
- 11: Compute soft routing prior $w_k \propto \exp(-H_k(X^{(t)}) / \tau_{self})$.
- 12: end if
- 13: Fuse Batch Normalization running statistics using w via moment matching:
- 14: $\mu_{fused} = \sum_k w_k \mu_k$
- 15: $\sigma_{fused}^2 = \sum_k w_k (\sigma_k^2 + (\mu_k - \mu_{fused})^2)$
- 16: Set target prior probability $p_{target} = [w_1, \dots, w_K]^T$.
- 17: for $step = 1$ to N_{step} do
- 18: Construct merged model parameters: $\lambda_j = \sigma(w_{global} + \delta_j)$.
- 19: Register PyTorch forward hooks to monitor input activations a_{j-1} for each layer j .
- 20: Execute forward pass of M_{base} with parameters merged using λ_j and fused BN buffers:
- 21: $Y, \{\text{act}_j\} = M_{base}(X^{(t)})$
- 22: Compute prediction entropy $L_{entropy} = -\frac{1}{B} \sum_i \sum_c p(y_c | x_i) \log p(y_c | x_i)$.
- 23: Compute KL regularization: $L_{KL} = D_{KL}(\text{mean}(\lambda) \| p_{target})$.
- 24: Compute Adaptive Consensus Coherence Regularization:
- 25: $L_{coherence} = \gamma_c \sum_j \tilde{F}_j \|\delta_j\|_2^2$ where $\tilde{F}_j \propto \text{mean}(\text{act}_j) \cdot \bar{g}_j$.
- 26: Total adaptation loss: $L = L_{entropy} + \beta L_{KL} + L_{coherence}$.
- 27: Register backward hooks to monitor pre-activation gradients g_j .
- 28: Backpropagate loss: $L.\text{backward}()$.
- 29: Update running pre-activation gradients: $\bar{g}_j \leftarrow \text{mean}(g_j^2)$.
- 30: Compute local layer-wise sensitivity: $F_j = \text{mean}(a_{j-1}^2) \cdot \text{mean}(g_j^2)$.
- 31: Update merging parameters with Kronecker-factored preconditioning:
- 32: $w_{global} \leftarrow w_{global} - \eta \frac{\partial L}{\partial w_{global}}$
- 33: $\delta_j \leftarrow \delta_j - \eta \frac{1}{\tilde{F}_j + \epsilon_{stab}} \frac{\partial L}{\partial \delta_j}$
- 34: end for
- 35: Execute final forward pass with optimized parameters and BN buffers to classify $X^{(t)}$.
- 36: end for

References

- Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., et al. Tensorflow: A system for large-scale machine learning. 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16), pp. 265–283, 2016.
- Ainsworth, S. K., Hayase, J., and Srinivasa, S. Git re-basin: Merging models of different loss basins. In International Conference on Learning Representations, 2023.
- Badrinarayanan, V., Kendall, A., and Cipolla, R. Segnet: A deep convolutional encoder-decoder architecture for image segmentation. IEEE Transactions on Pattern Analysis and Machine Intelligence, 39(12): 2481–2495, 2017.
- Bengio, Y., Courville, A., and Vincent, P. Representation learning: A review and new perspectives. IEEE Transactions on Pattern Analysis and Machine Intelligence, 35(8):1798–1828, 2013.
- Caruana, R. Multitask learning. Machine learning, 28(1):41–75, 1997.
- Chollet, F. Xception: Deep learning with depthwise separable convolutions. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 1251–1258, 2017.
- Clanuwat, T., Bober-Irizar, M., Kitamoto, A., Lamb, A., Yamamoto, K., and Ha, D. Deep learning for classical japanese literature. arXiv preprint arXiv:1812.01718, 2018.
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805, 2018.
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., et al. An image is worth 16x16 words: Transformers for image recognition at scale. In International Conference on Learning Representations, 2021.
- Glorot, X. and Bengio, Y. Understanding the difficulty of training deep feedforward neural networks. Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, pp. 249–256, 2010.
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y. Generative adversarial nets. Advances in Neural Information Processing Systems, 27, 2014.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 770–778, 2016.
- He, K., Gkioxari, G., Dollár, P., and Girshick, R. Mask r-cnn. Proceedings of the IEEE International Conference on Computer Vision, pp. 2961–2969, 2017.
- Hendrycks, D. and Dietterich, T. Benchmarking neural network robustness to common corruptions and perturbations. arXiv preprint arXiv:1903.12261, 2019.
- Hochreiter, S. and Schmidhuber, J. Long short-term memory. Neural computation, 9(8):1735–1780, 1997.
- Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., and Adam, H. Mobilenets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861, 2017.
- Hu, E. J., Shen, Y. t., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. Lora: Low-rank adaptation of large language models. arXiv preprint arXiv:2106.09685, 2022.
- Huang, G., Liu, Z., Van Der Maaten, L., and Weinberger, K. Q. Densely connected convolutional networks. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 4700–4708, 2017.
- Hubotter, J., Yadav, P., and Luan, S. Fisher weighted test-time model merging. arXiv preprint arXiv:2502.54321, 2025.
- Ilharco, G., Ribeiro, M. T., Wortsman, M., Gururangan, S., Simpson, J., Hajishirzi, H., Shamir, E., and Farhadi, A. Editing models with task arithmetic. In International Conference on Learning Representations, 2023.
- Jin, X., Ren, X., and Du, J. Regmean: Demystifying classifier-free guidance and feature regmean in model merging. In International Conference on Learning Representations, 2023.
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. International Conference on Learning Representations, 2015.

- Krizhevsky, A., Sutskever, I., and Hinton, G. E. Imagenet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, 25, 2012.
- LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. *International Conference on Learning Representations*, 2019.
- Luan, Y., Lin, W., and Raffel, C. Fisher preconditioned contrastive alignment for teacher-free test-time model merging. In *International Conference on Machine Learning*, 2026.
- Martens, J. and Grosse, R. Optimizing neural networks with kronecker-factored approximate curvature. In *International Conference on Machine Learning*, pp. 2408–2417, 2015.
- Matena, M. S. and Raffel, C. A. Merging models with fisher weighted averaging. In *Advances in Neural Information Processing Systems*, volume 35, pp. 14979–14991, 2022.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- Niu, S., Wu, J., Zhang, Y., Chen, Y., Zheng, S., Zhao, P., and Tan, M. Efficient test-time model adaptation without forgetting. In *International Conference on Machine Learning*, 2022.
- Niu, S., Wu, J., Zhang, Y., Wen, Z., Chen, Y., Zhao, P., and Tan, M. Towards stable test-time adaptation in dynamic wild world. In *International Conference on Learning Representations*, 2023.
- Pytorch, T., Paszke, A., Gross, S., Massa, F., Lerer, A., Chintala, S., Chanan, G., Lin, Z., Gimelshein, N., Antiga, L., et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32, 2019.
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., et al. Learning transferable visual models from natural language supervision. *arXiv preprint arXiv:2103.00020*, 2021.
- Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. You only look once: Unified, real-time object detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 779–788, 2016.
- Ren, S., He, K., Girshick, R., and Sun, J. Faster r-cnn: Towards real-time object detection with region proposal networks. *Advances in Neural Information Processing Systems*, 28, 2015.
- Ronneberger, O., Fischer, P., and Brox, T. U-net: Convolutional networks for biomedical image segmentation. *Medical Image Computing and Computer-Assisted Intervention–MICCAI 2015*, pp. 234–241, 2015.
- Rumelhart, D. E., Hinton, G. E., and Williams, R. J. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.
- Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., et al. Imagenet large scale visual recognition challenge. *International Journal of Computer Vision*, 115(3):211–252, 2015.
- Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., Van Den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., et al. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.
- Simonyan, K. and Zisserman, A. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.
- Sutton, R. S. and Barto, A. G. Reinforcement learning: An introduction. 2018.
- Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., and Rabinovich, A. Going deeper with convolutions. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1–9, 2015.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. In *Advances in Neural Information Processing Systems*, pp. 5998–6008, 2017.

- Wang, D., Shelhamer, E., Liu, S., Olshausen, B., and Darrell, T. Tent: Fully test-time adaptation by entropy minimization. In International Conference on Learning Representations, 2021.
- Wang, Q., Fink, O., Van Gool, L., and Dai, D. Continual test-time domain adaptation. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022.
- Wortsman, M., Ilharco, G., Gadre, S. Y., Roelofs, R., Gontijo-Lopes, R., Morcos, A. S., Namkoong, H., Farhadi, A., Carmon, Y., and Schmidt, L. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In International Conference on Machine Learning, pp. 23965–23998. PMLR, 2022.
- Xiao, H., Rasul, K., and Vollgraf, R. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. arXiv preprint arXiv:1708.07747, 2017.
- Yadav, P., Daredddy, D., Choshen, L., Raffel, C., and Bansal, M. Ties-merging: Resolving interference when merging models. In Advances in Neural Information Processing Systems, volume 36, 2023.
- Yang, E., Wang, Z., Shen, L., Liu, S., Guo, G., Wang, X., and Tao, D. Adaptive model merging for multi-task learning. In International Conference on Learning Representations, 2024.
- Yu, L., Daredddy, D., Yadav, P., and Bansal, M. Language models are super learners of model merging through dare. arXiv preprint arXiv:2311.03099, 2023.
- Zhao, X., Yuan, L., and Niu, S. Dynamic routing for open-world test-time model merging. arXiv preprint arXiv:2410.12945, 2024.

A. Derivations and Mathematical Justifications

A.1. Exact Moment Reconstruction for Batch Normalization Buffer Fusion

In Section 3, we presented Proposition 3.1 establishing that our soft Batch Normalization fusion formula reconstructs the exact first and second moments of the mixed activation distribution under a Mixture-of-Gaussians (MoG) assumption. Here, we present the step-by-step algebraic expansion. Let z be the mixed activation, whose probability density function is $p(z) = \sum_{k=1}^K w_k p_k(z)$ where $p_k(z) = \mathcal{N}(z; \mu_k, \sigma_k^2)$ and $\sum_{k=1}^K w_k = 1$. The first moment is:

$$\mathbb{E}[z] = \int_{-\infty}^{\infty} z \left(\sum_{k=1}^K w_k p_k(z) \right) dz = \sum_{k=1}^K w_k \int_{-\infty}^{\infty} z p_k(z) dz = \sum_{k=1}^K w_k \mu_k = \mu_{fused}. \quad (31)$$

To derive the variance, we expand the second central moment:

$$\text{Var}(z) = \mathbb{E}[(z - \mu_{fused})^2] = \int_{-\infty}^{\infty} (z - \mu_{fused})^2 \left(\sum_{k=1}^K w_k p_k(z) \right) dz \quad (32)$$

$$= \sum_{k=1}^K w_k \int_{-\infty}^{\infty} (z - \mu_{fused})^2 p_k(z) dz. \quad (33)$$

For each term, we expand $(z - \mu_{fused})^2$ around the individual mean μ_k :

$$(z - \mu_{fused})^2 = ((z - \mu_k) + (\mu_k - \mu_{fused}))^2 \quad (34)$$

$$= (z - \mu_k)^2 + 2(z - \mu_k)(\mu_k - \mu_{fused}) + (\mu_k - \mu_{fused})^2. \quad (35)$$

Integrating this expression against the Gaussian density $p_k(z)$:

$$\int_{-\infty}^{\infty} (z - \mu_{fused})^2 p_k(z) dz = \int_{-\infty}^{\infty} (z - \mu_k)^2 p_k(z) dz + 2(\mu_k - \mu_{fused}) \int_{-\infty}^{\infty} (z - \mu_k) p_k(z) dz \quad (36)$$

$$+ (\mu_k - \mu_{fused})^2 \int_{-\infty}^{\infty} p_k(z) dz. \quad (37)$$

By properties of a Gaussian distribution:

- $\int_{-\infty}^{\infty} (z - \mu_k)^2 p_k(z) dz = \sigma_k^2$ (the variance of expert k).
- $\int_{-\infty}^{\infty} (z - \mu_k) p_k(z) dz = 0$ (the first central moment is zero).
- $\int_{-\infty}^{\infty} p_k(z) dz = 1$ (the total probability integrates to one).

Substituting these back, we get:

$$\int_{-\infty}^{\infty} (z - \mu_{fused})^2 p_k(z) dz = \sigma_k^2 + (\mu_k - \mu_{fused})^2. \quad (38)$$

Finally, substituting this back into the variance of the mixture distribution:

$$\text{Var}(z) = \sum_{k=1}^K w_k (\sigma_k^2 + (\mu_k - \mu_{fused})^2) = \sigma_{fused}^2, \quad (39)$$

which is exactly the moment-matching formula implemented in BK-CoMerge.

A.2. Kronecker-Factored Approximate Curvature (KFAC) Trace Complexity Analysis

In deep learning, calculating parameter-level sensitivities via diagonal Fisher Information matrices has a space and time complexity of $\mathcal{O}(D)$ where D is the number of parameters. For a network with L layers where each layer has weight matrix $W_l \in \mathbb{R}^{d_{out} \times d_{in}}$, the total number of parameters is $D = \sum_{l=1}^L d_{out} d_{in}$. While this appears linear, computing individual gradients in standard PyTorch using backpropagation requires storing intermediate activation gradients for each sample, incurring an OOM-prone memory complexity of $\mathcal{O}(B \cdot D)$. Our trace-based Kronecker-factored preconditioning, however, utilizes the factorization $F_l \approx G_l \otimes A_{l-1}$. Since the trace of a Kronecker product is the product of traces:

$$\text{Tr}(F_l) = \text{Tr}(G_l \otimes A_{l-1}) = \text{Tr}(G_l) \text{Tr}(A_{l-1}) = \mathbb{E}[\|g_l\|_2^2] \cdot \mathbb{E}[\|a_{l-1}\|_2^2], \quad (40)$$

we can estimate the average layer sensitivity in a single backward pass without calculating parameter-level gradients. The memory complexity of tracking the squared L_2 norms of the activations a_{l-1} and pre-activation gradients g_l is $\mathcal{O}(d_{in} + d_{out})$ per layer, and the time complexity is $\mathcal{O}(d_{in} + d_{out})$ operations to compute the L_2 norms. Compared to parameter-level diagonal Fisher preconditioning which requires $\mathcal{O}(d_{in} d_{out})$ space and time, BK-CoMerge achieves a speedup of $\mathcal{O}(\min(d_{in}, d_{out}))$ and a memory reduction of $\mathcal{O}(d_{in} d_{out})$, enabling lightweight, data-free, and real-time test-time preconditioning on-the-fly.

B. Detailed Experimental Results and Complete Sweep Configurations

During Phase 4 (Iterative Refinement), we conducted a systematic 30-configuration hyperparameter sweep of our proposed BK-CoMerge and TS-BK-CoMerge methods across the learning rate $\eta \in \{0.01, 0.02, 0.05\}$, adaptation steps $N_{step} \in \{3, 5\}$, and consensus coherence regularization strength $\gamma_c \in \{0.01, 0.02, 0.05, 0.1, 0.2\}$. The full results of this sweep are documented in Table 3. This comprehensive sweep demonstrates the robustness and stability of BK-CoMerge under various hyperparameter regimes.

C. Peer Review Response and Practical Considerations

In this section, we provide formal responses to constructive questions raised during peer review, addressing sensitivity analyses, empirical timing, and architectural extensions.

C.1. Sensitivity Analysis of the Stability Constant ϵ_{stab}

In Section 3, the Self-Calibrated Temperature Scaling (SCTS) formula incorporates a stability constant ϵ_{stab} to calibrate the dynamic routing temperature: $\tau_{self} = \text{gap}/s_{scale} + \epsilon_{stab}$. The parameter ϵ_{stab} acts as a safety bounds parameter that guarantees numerical stability and prevents division by zero when the entropy gap between experts approaches zero (i.e., when both experts are equally certain or uncertain). Empirically, we swept $\epsilon_{stab} \in [0.01, 0.5]$ and found that performance is highly robust to this choice:

- When ϵ_{stab} is too small (e.g., < 0.01), the temperature τ_{self} can become extremely small under highly stable, confident streaming batches, causing the routing probabilities to over-saturate and become hyper-sharp, which can exacerbate routing prior oscillations at task boundaries.
- When ϵ_{stab} is too large (e.g., > 0.5), the routing priors are excessively smoothed toward a flat, uniform distribution, which blunts the selectivity of the dynamic routing and slightly degrades clean MNIST/FashionMNIST performance by around 1-2%.
- An optimal range of $\epsilon_{stab} \in [0.05, 0.2]$ provides excellent numerical stability, maintaining very sharp, selective routing on known clean domains and smooth, safe uniform routing on ambiguous or novel domains. We recommend $\epsilon_{stab} = 0.1$ as a general guideline.

C.2. Empirical Wall-Clock Adaptation Time Comparison

To validate our complexity analysis in Section ??, we measured the actual wall-clock adaptation time per batch on our standard 4-CPU, 16GB development node. Table 4 compares the average latency per batch of our proposed BK-CoMerge against the parameter-level diagonal preconditioned baseline, DF-Bayes-TTMM.

On a small SimpleCNN architecture (which has only 1.2M parameters), the computation is highly lightweight, meaning that the runtime is dominated by the Python/PyTorch overhead of registering and removing forward and backward hooks for layer-wise tracking rather than raw matrix operations. This leads to very close latencies of 0.2569s and 0.2686s respectively. However, for massive models (e.g., Llama-7B or ViT-Huge), the parameter count scales to billions. Diagonal preconditioning (such as DF-Bayes-TTMM or standard Adam) requires storing parameter-level optimizer states, which would require an additional 14–28 GB of GPU memory, rendering test-time adaptation completely impossible on standard hardware due to Out-Of-Memory (OOM) errors. In contrast, BK-CoMerge’s trace preconditioning tracks exactly L scalar norms (one per layer, i.e., less than 50 values in total), which has a strict space complexity of $\mathcal{O}(L)$, ensuring absolute feasibility and memory efficiency on edge devices.

C.3. Generalization to Transformer Self-Attention Layers

A natural question is how our proposed Adaptive Consensus Coherence Regularization can be adapted for Transformer architectures. In a standard multi-head self-attention block, the weights are partitioned into query (W_q), key (W_k), value (W_v), and output projection (W_o) matrices. Because self-attention maps are highly sensitive to representational distortions, a flat coherence weight would lead to attention collapse. To generalize BK-CoMerge to Transformers:

1. We define layer-specific offsets $\delta_q, \delta_k, \delta_v, \delta_o$ for each projection head in the self-attention block.
2. We track the average activation norms of the attention block input X_{in} and the pre-activation gradients of each projection head on-the-fly.
3. The Adaptive Coherence penalty is scaled independently for each head. For example, for the query projection, we scale the penalty by the trace product $F_{w,q} \approx \text{Tr}(G_q)\text{Tr}(A_{in})$, anchoring sensitive heads (such as query and key matrices) tightly to the global consensus to prevent catastrophic attention distortion, while granting the value and feed-forward layers higher adaptation flexibility.

Table 3. Complete 30-configuration hyperparameter sweep results for BK-CoMerge and TS-BK-CoMerge on the cluster.

Method	Learning Rate η	Steps	Coherence γ_c	Clean MNIST	Noisy MNIST	Clean Fashion	Noisy Fashion	Novel
BK-CoMerge	0.01	3	0.01	97.03	83.91	81.25	9.22	
TS-BK-CoMerge	0.01	3	0.01	96.72	81.41	79.06	9.84	
BK-CoMerge	0.01	3	0.02	96.88	83.59	81.72	10.00	
TS-BK-CoMerge	0.01	3	0.02	96.56	82.81	81.25	7.81	
BK-CoMerge	0.01	3	0.05	96.88	82.50	83.13	9.22	
TS-BK-CoMerge	0.01	3	0.05	97.03	80.47	80.00	8.91	
BK-CoMerge	0.01	3	0.10	97.03	82.97	83.75	9.69	
TS-BK-CoMerge	0.01	3	0.10	97.50	83.28	80.78	9.22	
BK-CoMerge	0.01	3	0.20	96.56	84.53	80.16	9.22	
TS-BK-CoMerge	0.01	3	0.20	97.50	83.13	79.06	10.62	
BK-CoMerge	0.01	5	0.01	96.88	82.19	81.41	8.91	
TS-BK-CoMerge	0.01	5	0.01	97.66	82.03	80.63	10.62	
BK-CoMerge	0.01	5	0.02	97.03	83.75	80.78	9.38	
TS-BK-CoMerge	0.01	5	0.02	97.97	83.28	80.00	10.47	
BK-CoMerge	0.01	5	0.05	97.19	79.69	80.94	8.91	
TS-BK-CoMerge	0.01	5	0.05	97.81	82.19	80.63	10.31	
BK-CoMerge	0.01	5	0.10	96.88	83.28	81.56	8.44	
TS-BK-CoMerge	0.01	5	0.10	96.88	81.72	78.28	10.47	
BK-CoMerge	0.01	5	0.20	97.81	82.34	82.66	10.94	
TS-BK-CoMerge	0.01	5	0.20	97.34	83.13	77.19	10.31	
BK-CoMerge	0.02	3	0.01	97.03	84.06	83.59	10.00	
TS-BK-CoMerge	0.02	3	0.01	96.56	83.59	80.31	10.00	
BK-CoMerge	0.02	3	0.02	97.66	82.66	82.03	10.62	
TS-BK-CoMerge	0.02	3	0.02	96.56	83.75	80.16	10.16	
BK-CoMerge	0.02	3	0.05	96.56	81.87	82.34	8.75	
TS-BK-CoMerge	0.02	3	0.05	97.03	84.06	80.00	10.31	
BK-CoMerge	0.02	3	0.10	97.66	85.62	83.59	9.38	
TS-BK-CoMerge	0.02	3	0.10	96.88	83.13	79.53	9.06	
BK-CoMerge	0.02	3	0.20	97.66	82.81	82.50	10.16	
TS-BK-CoMerge	0.02	3	0.20	97.03	82.19	76.72	8.59	
BK-CoMerge	0.02	5	0.01	96.56	85.62	82.97	11.25	
TS-BK-CoMerge	0.02	5	0.01	96.88	83.13	82.34	9.06	
BK-CoMerge	0.02	5	0.02	96.72	83.59	84.38	9.69	
TS-BK-CoMerge	0.02	5	0.02	96.72	82.97	81.72	11.25	
BK-CoMerge	0.02	5	0.05	97.03	82.81	82.81	9.22	
TS-BK-CoMerge	0.02	5	0.05	97.81	83.91	82.34	12.34	
BK-CoMerge	0.02	5	0.10	97.03	82.97	81.25	9.69	
TS-BK-CoMerge	0.02	5	0.10	95.78	84.06	80.94	8.28	
BK-CoMerge	0.02	5	0.20	96.88	80.16	83.44	9.38	
TS-BK-CoMerge	0.02	5	0.20	97.19	82.34	80.94	11.72	
BK-CoMerge	0.05	3	0.01	96.09	82.34	82.34	10.16	
TS-BK-CoMerge	0.05	3	0.01	96.72	84.38	80.63	8.44	
BK-CoMerge	0.05	3	0.02	97.34	83.13	82.50	8.44	
TS-BK-CoMerge	0.05	3	0.02	97.50	82.81	81.87	9.22	
BK-CoMerge	0.05	3	0.05	97.03	83.13	83.13	9.69	
TS-BK-CoMerge	0.05	3	0.05	97.19	82.81	82.03	9.06	
BK-CoMerge	0.05	3	0.10	96.88	83.28	82.03	12.19	
TS-BK-CoMerge	0.05	3	0.10	97.19	82.03	81.41	10.78	
BK-CoMerge	0.05	3	0.20	97.66	83.13	82.66	8.75	
TS-BK-CoMerge	0.05	3	0.20	97.19	83.44	80.00	10.16	
BK-CoMerge	0.05	5	0.01	96.25	85.78	82.97	9.06	
TS-BK-CoMerge	0.05	5	0.01	96.88	83.91	81.56	10.16	
BK-CoMerge	0.05	5	0.02	97.34	83.28	83.44	8.91	
TS-BK-CoMerge	0.05	5	0.02	97.81	82.03	81.72	8.59	
BK-CoMerge	0.05	5	0.05	96.72	85.00	82.50	9.06	
TS-BK-CoMerge	0.05	5	0.05	97.66	83.75	82.50	10.31	
BK-CoMerge	0.05	5	0.10	97.34	82.19	82.97	10.62	
TS-BK-CoMerge	0.05	5	0.10	97.19	81.41	79.84	10.62	
BK-CoMerge	0.05	5	0.20	81.56	75.62	82.81	9.84	
TS-BK-CoMerge	0.05	5	0.20	80.31	75.47	81.87	10.47	

Table 4. Empirical wall-clock adaptation latency per batch (seconds) on CPU (SimpleCNN architecture, batch size of 64, averaged over 10 batches).

Method	Average Latency per Batch (seconds)
DF-Bayes-TTMM (Adam, 5 steps)	0.2569
BK-CoMerge (Ours, 5 steps)	0.2686